

Device Status Heartbeat Uplink Protocol

Table of Contents

- 0. Integration Checklist
- 1. Message
- 2. Fields
 - 2.1 Envelope
 - 2.2 timezone — context for interpreting ts
 - 2.3 business — business context
 - 2.4 network — topology state
- 3. Conventions and Boundaries
 - 3.1 Empty object $\neq$ missing key
 - 3.2 slaves is the full configured topology, not "the ones currently connected"
 - 3.3 network and faults do not overlap
 - 3.4 this stream's timezone is "the local device's", the other two streams' is "the master's"
 - 3.5 Offline judgment: no intermediate state
 - 3.6 Failure behavior: no resend, no back-fill, no store, no backoff
 - 3.7 Transport details
- 4. Reconciliation with the Two Data Streams
- 5. Common Misuse Checklist

Device $\rightarrow$ customer server. Each device independently POSTs; the customer server only implements one receiving endpoint.

In one sentence: each device periodically reports a full current-state snapshot. Received = online, not received = offline, no intermediate state.

It is a different matter from the underlying data uplink protocol (flow_events / reid_persons_delta of upload_stream_api.md); the two are independent:

	Heartbeat	Two underlying data streams
Who sends	Each device sends its own (master, client, or standalone)	Only the master sends, on behalf of the whole group
Content	Full current-state snapshot at this moment	Business data, incremental
On failure	Discard; send a new snapshot next period	Must arrive; resend on failure
Value of late arrival	None — old state is wrong state	Has value — data does not expire due to lateness

Whether a device is online is answered only by this protocol. Silence of the data stream does not mean the device is offline (no flow means no message), and a stalled data-stream water level does not mean the device is offline.

0. Integration Checklist

Implementing the receiver needs only the five things below; the rest of the document explains fields and edge cases, consult as needed.

Purpose	Method & default path	Content-Type
Heartbeat message	POST /api/passenger-flow/device-status	application/json

1. Return 2xx, and the response body must be non-empty (even just `{}`). The device treats "2xx but empty body" as failure — a reverse proxy or gateway may return an empty 200 when the backend never received the request. This is the same criterion as the two data streams, so the server can reuse one response logic.

2. Aggregate by the `sn` in the message. Each device has its own address and sends its own; messages have no subordinate relationship, and the server need not know who is whose master — the topology is described by the message itself (§2.4).

3. The message is a full snapshot; overwrite the whole record. No incremental, no field-level patch, no sequence number — naturally idempotent, duplicate delivery is harmless.

4. Online judgment = whether it arrived on time. Do not introduce a threshold like "offline only after N consecutive losses" (§3.5).

5. A non-contiguous message sequence is normal. During network outage those few periods simply have no message; after recovery the device sends the current snapshot directly, without back-filling intermediate state (§3.6).

On receiving a message:

```
device[message.sn] = full message text; last_arrival_time = now    # overwrite
whole, no merge
return 200 {"ok": true}
```

Online judgment (a timer the server runs itself, unrelated to the message):

```
now - device[sn].last_arrival_time > period_s + network_margin ⇒ offline
```

The server address, path, and reporting period are all in each device's own configuration, adaptable to a gateway prefix; the config interface is in `http_device_status_api.md` §2. The upload address is neither shared with nor falls back to the two data streams — configuring the data stream will not make the heartbeat start reporting, and vice versa.

1. Message

The message shape is determined by `network\_role`, three kinds in total. The top-level keys of all three shapes are identical (only the content of `network` differs), so the consumer needs only one parser.

Master station (network_role: "master")

```
{
  "schema_version": "1.0",
```

```

    "type": "device_status",
    "sn": "211000002604150751",
    "network_role": "master",
    "ts": 1786008767,
    "timezone": { "name": "Asia/Shanghai", "utc_offset_seconds": 28800, "is_dst":
false },
    "business": {
        "batch_generation": 1785995727,
        "batch_date": "2026-08-06",
        "state": "opening"
    },
    "network": {
        "slaves": [
            { "sn": "211000002604150752", "connected": true, "ip": "192.168.1.22" },
            { "sn": "211000002604150753", "connected": false, "ip": "" }
        ]
    },
    "faults": []
}

```

Client station (network_role: "client")

```

{
    "schema_version": "1.0",
    "type": "device_status",
    "sn": "211000002604150752",
    "network_role": "client",
    "ts": 1786008767,
    "timezone": { "name": "Asia/Shanghai", "utc_offset_seconds": 28800, "is_dst":
false },
    "business": {
        "batch_generation": 1785995727,
        "batch_date": "2026-08-06",
        "state": "opening"
    },
    "network": {
        "master": { "sn": "211000002604150751", "connected": false, "reason":
"network_error" }
    },
    "faults": []
}

```

Standalone (network_role: "off")

```

{
    "schema_version": "1.0",
    "type": "device_status",
    "sn": "211000002604150754",
    "network_role": "off",
    "ts": 1786008767,
    "timezone": { "name": "Asia/Shanghai", "utc_offset_seconds": 28800, "is_dst":
false },
    "business": {
        "batch_generation": 1785995727,

```

```

    "batch_date": "2026-08-06",
    "state": "opening"
  },
  "network": {},
  "faults": []
}

```

2. Fields

2.1 Envelope

Eight top-level items; keys always present:

Field	Type	Value / meaning
schema_version	string	Always <code>"1.0"</code>
type	string	Always <code>"device_status"</code>
sn	string	This device's serial number. The server aggregates by it; it is this device's identity
network_role	string	master client off (master station / client station / standalone). After the role changes at runtime, the next period's message shape follows
ts	uint64	Message generation moment, 10-digit Unix seconds (UTC epoch seconds)
timezone	object	This device's time zone, see 2.2; empty object <code>{}</code> when unavailable
business	object	Business context, see 2.3; empty object <code>{}</code> when unavailable
network	object	Topology state, see 2.4; empty object <code>{}</code> for standalone
faults	array	The device's own anomalies. Currently always an empty array <code>[]</code> (reserved, see 3.3)

2.2 timezone — context for interpreting ts

Three keys always present; the whole block is an empty object `{}` when unavailable. Same shape as the `timezone` block of the two data streams; the three streams can share one parser.

Key	Type	Meaning
name	string	Authoritative value — only the time-zone name lets you recompute a historical day's DST rules
utc_offset_seconds	int32	Offset seconds from UTC at

		the moment of packaging (east positive, +08:00 → 28800), for servers without tzdata to render local time directly
is_dst	bool	Whether this moment is in DST; can be cross-checked with `name`; a mismatch indicates a device time-zone config problem

⚠ `name` is not guaranteed to be an IANA time-zone name. The device may be set directly by offset, in which case `name` looks like `"UTC-12:45"`. Accept both forms; do not discard the message just because IANA time-zone-library parsing failed.

⚠ The offset is a snapshot, not a constant of that device. In DST regions the same device reports two different `utc\_offset\_seconds` within a year.

This stream's time zone is the local device's; the other two streams' is the master station's — this distinction is useful, see 3.4.

2.3 business — business context

Field	Type	Value / meaning
batch_generation	int64	Current batch generation, same source and value as the same-named field in the two data streams' envelope, see section 4
batch_date	string	Current business day YYYY-MM-DD, same as above, same source and value
state	string	before_open (not yet open) opening (open) after_close (closed)

When business-period info is temporarily unavailable the whole block is an empty object `{}`; the rest of the message is normal.

⚠ `batch\_date` is the business day, divided by the device's reset_time, and may cross midnight — with reset_time 02:00, the heartbeat at 01:00 still reports the previous day. It is not necessarily the same as the natural day derived from `ts`; this is not an error.

2.4 network — topology state

Always present; shape determined by `network\_role`: master gives `slaves`, client gives `master`, standalone is an empty object `{}`.

Field	Type	Value / meaning
slaves[]	array	Appears only in master messages. The full configured topology, including slaves that never connected (see 3.2). Empty array when zero slaves

slaves[].sn	string	Slave serial number
slaves[].connected	bool	Whether this slave currently maintains connection with this master
slaves[].ip	string	The slave's connection source IP; empty string ```` when not connected
master	object	Appears only in client messages. This device's connection status with the master. Key absent when unavailable (network is empty object), do not fabricate
master.sn	string	Master serial number, from this device's network config
master.connected	bool	Whether this device currently maintains connection with the master
master.reason	string	Reason for not connected, see table below

``master.reason`` vocabulary (closed):

Value	Meaning	On-site handling
none	No anomaly (the value when connected: true)	—
network_error	Network layer unreachable (disconnect, timeout, master process not running)	Check cabling / check whether master is alive
rejected	Connected, but master refused access	Check master-side slave admission config
sn_mismatch	Connected, but serial number does not match the master's configured list	Config error: verify master's slave list against this device's SN

The latter two have completely different on-site handling from `network_error`: the line is up, the config is wrong. The alert text is worth separating.

3. Conventions and Boundaries

3.1 Empty object ≠ missing key

When ``timezone`` / ``business`` / ``network`` are unavailable, what is given is an empty object `{}``, meaning "checked, this device has no such item right now"; whereas a missing key means "this device version does not yet report this item". The two must be distinguishable, otherwise troubleshooting is impossible when device versions are mixed. Do not treat empty object and missing key the same.

3.2 slaves is the full configured topology, not "the ones currently connected"

The master message's ``slaves`` lists all slaves configured on the master, including those that never connected: unconnected entries are still in the array, just with ``connected: false``, ``ip: ````.

This is deliberate: if you list only online ones, the customer can never see "5 configured, now only 3 left" — and that is exactly the case most worth alerting on. The alert logic should be based on ``connected: false`` entries, not on array-length changes.

3.3 network and faults do not overlap

``network`` is the topology relationship — a resident state, always has a value at any moment.

``faults`` is the device's own health issues — normally an empty array.

"A slave cannot reach the master" belongs to the former and will not simultaneously appear in ``faults``; the consumer need not do cross-field deduplication.

``faults`` is a reserved field: currently always an empty array; the device's own health issues (disk, temperature, camera, algorithm, etc.) will be reported through it in the future. The field is already in the message now, so the consumer's parsing structure need not change version for the first fault — parse it as an array now and tolerate non-empty; do not assert it is always empty.

3.4 this stream's timezone is "the local device's", the other two streams' is "the master's"

Each device reports its own heartbeat, so ``timezone`` is that device's own time zone; whereas `flow_events` / `reid_persons_delta` are reported by the master on behalf of the whole group, and the time zone in those two streams is the master's (`batch_date` is determined by the master's time zone for the day).

This distinction is useful: in a normal topology all devices in the group are in one store and must share one time zone, so if the ``timezone`` reported by each heartbeat is inconsistent, it means some device in the group has had its time zone misconfigured — which is itself a config incident worth alerting on. The other two streams cannot do this; they only have the master's one copy.

3.5 Offline judgment: no intermediate state

Received = online, not received = offline. The device side defines no "tolerance window", and does not recommend the server introduce a threshold like "offline only after N consecutive losses" — that would create an intermediate state for "online", and the whole value of the heartbeat is precisely that it has no intermediate state.

Worst-case delay from a state change to customer-visible: one reporting period (default 60 seconds).

To be more sensitive, the only knob is the device-side ``period_s`` (lower bound 5 seconds). The judgment side has no, and should have no, adjustable threshold.

⚠ Do not blindly turn ``period_s`` to the lower bound: each device connects directly, so the request volume the server bears = $\text{device_count} \times 60 \div \text{period_s}$ per minute. The default 60 seconds is enough for the vast majority of scenarios.

3.6 Failure behavior: no resend, no back-fill, no store, no backoff

On send failure the device does nothing, and sends a latest snapshot next period. Reason: the value of state information decays over time — the "connection dropped" from 5 minutes ago is meaningless to the customer, and may well have recovered by now.

This yields three properties the consumer needs to know:

The message may be missing. The consumer should not assume the message sequence is contiguous, nor expect back-fill after recovery.

A connection-state change does not trigger an extra report. Only the periodic send path exists: a client that disconnects and recovers within one period will only show consecutive ``connected: true`` on the customer side, and will not receive the pair of noise messages "disconnected" and "recovered". This is deliberate — small issues can self-heal within one period, while real faults persist and will certainly be reported next period.

Failure does not make the device send faster: the period is counted from "the moment of sending", independent of success or failure. During server unreachability there is no retry storm; after recovery it naturally resumes from the next due point.

3.7 Transport details

Single-request timeout 10 seconds.

HTTPS optional (device-side config), does not validate the server certificate, self-signed certificates can be used directly.

When the address is not configured (host empty or port 0) the device sends no heartbeat, and throttles the device log to one alert per 60 seconds. The first message is sent immediately after the address is configured; those not sent before are not back-filled.

4. Reconciliation with the Two Data Streams

``business.batch_generation`` and ``business.batch_date`` share the same source and value as the same-named fields in the following places:

- The envelope of `flow_events` / `reid_persons_delta` uplink messages (see `upload_stream_api.md` section 4)
- `GET /api/passenger-flow/date/reid`, `GET /api/passenger-flow/date/flow-summary`

Therefore the heartbeat can directly answer two questions the data side cannot:

- "Which batch is this device currently writing data into?" — just reconcile with the already-received flow/ReID data; after a baseline reset (model change) the heartbeat reflects the new ``batch_generation`` in the next period.
- "No data for a period — no one, or device down?" — heartbeat normal but data stream silent $\Rightarrow$ that period truly had no flow (the data stream sends no message for empty windows); heartbeat also down $\Rightarrow$ device disconnected. This is the only way to distinguish the two cases.

⚠ The heartbeat only reports "current batch", and does not promise where the data has been uploaded to. The data's integrity water level is ``complete_through_ts`` in the `flow_events` envelope; the two answer different questions, do not mix them.

5. Common Misuse Checklist

Misuse	Correct approach	See
Return a 200 with empty body	Response body must be non-empty, or the device judges failure	0
Introduce a threshold like "offline only after 3 consecutive losses"	Sensitivity only adjusts `period_s`; the judgment side has no threshold	3.5
Alert "packet loss" when a few messages are missing	Failure does not back-fill; the sequence is inherently non-contiguous	3.6
Wait for a "disconnect" event message	Only periodic snapshots; jitter that self-heals within one period is invisible	3.6
Treat `network: {}` as "device did not report topology"	Empty object = checked and none; missing key means version unsupported	3.1
Alert on `slaves` array-length changes	Length is the full configured topology; look at `connected: false`	3.2
Go looking in `faults` for a slave going down	The two blocks do not overlap; topology issues are only in `network`	3.3
Assert `faults` is always empty, non-empty means parse failure	Parse as array and tolerate non-empty	3.3
Discard the message when IANA time-zone-library parsing of `name` fails	`name` may be in the form `"UTC-12:45"`; accept both	2.2
Use today's `utc_offset_seconds` to recompute history	In DST regions the offset changes; recompute via `name`	2.2
Use the heartbeat's `timezone` as the group's time zone	It is the local device's; inconsistency across devices is itself an alert item	3.4
Attribute `batch_date` by `ts`'s natural day	Business day may cross midnight; the two are not necessarily the same	2.3
Use data-stream silence or stalled water level to judge device offline	Online status is answered only by this protocol	4
Use `business.batch_generation` as "data has been delivered up to here"	The integrity water level is `complete_through_ts`	4